Implementation of Logic gates in Verilog

```
//file: and_gate.v

module and_gate(
    input a,
    input b,
    output y
);
    assign y = a & b;  // AND operation
endmodule



//file: or_gate.v

module or_gate(
    input a,
    input b,
    output y
);
    assign y = a | b;  // OR operation
endmodule



//file: not_gate.v

module not_gate(
    input a,
    output y
);
    assign y = ~a;  // NOT operation
endmodule
```

```verilog
//file: nand_gate.v

module nand_gate(
    input a,
    input b,
    output y
);
    assign y = ~(a & b);  // NAND operation
endmodule


//file: nor_gate.v

module nor_gate(
    input a,
    input b,
    output y
);
    assign y = ~(a | b);  // NOR operation
endmodule


//file: xor_gate.v

module xor_gate(
    input a,
    input b,
    output y
);
    assign y = a ^ b;  // XOR operation
endmodule
```

Explanation :

1. **module**: This keyword is used to define a module, which is a basic building block in Verilog. Here, xor_gate is the name of the module.
2. **input**: This keyword declares an input port to the module. In this code, a and b are input ports.

3. **output**: This keyword declares an output port from the module. In this code, y is an output port.
4. **assign**: This keyword is used to assign a value to a wire. It is used here to define the logic operation for the XOR gate.
5. **endmodule**: This keyword marks the end of a module definition.

```
//file: logic_gates_tb.v

`timescale 1ns / 1ps

module logic_gates_tb;
    reg a, b;  // Inputs for testing
    wire and_y, or_y, not_y, xor_y, nand_y, nor_y;

    // Instantiate the gate modules
    and_gate u1 (.a(a), .b(b), .y(and_y));
    or_gate u2 (.a(a), .b(b), .y(or_y));
    not_gate u3 (.a(a), .y(not_y));
    xor_gate u4 (.a(a), .b(b), .y(xor_y));
    nand_gate u5 (.a(a), .b(b), .y(nand_y));
    nor_gate u6 (.a(a), .b(b), .y(nor_y));

    // Test stimulus
    initial begin
        $display("Time | a | b | AND | OR | NOT(a) | XOR | NAND | NOR");
        $monitor("%4t | %b | %b |  %b  | %b  |   %b    |  %b  |   %b   |  %b",
                 $time, a, b, and_y, or_y, not_y, xor_y, nand_y, nor_y);

        // Test cases
        a = 0; b = 0; #10;
        a = 0; b = 1; #10;
        a = 1; b = 0; #10;
        a = 1; b = 1; #10;
        $finish;  // End the simulation
    end
endmodule
```

Explanation:

1. **timescale**: This directive specifies the time unit and time precision for the simulation. In this case, 1ns / 1ps means the time unit is 1 nanosecond and the time precision is 1 picosecond.
2. **module**: This keyword is used to define a module, which is a basic building block in Verilog. Here, logic_gates_tb is the name of the testbench module.
3. **reg**: This keyword declares a register, which is a storage element that can hold a value. In this code, a and b are declared as registers to hold the input values for testing.
4. **wire**: This keyword declares a wire, which is used to connect different parts of a circuit.
   Here, and_y, or_y, not_y, xor_y, nand_y, and nor_y are wires that hold the output values from the gate modules.
5. **initial**: This keyword is used to define an initial block, which contains statements that are executed once at the beginning of the simulation. The initial block is used to apply test stimuli to the circuit.
6. **$display**: This system task is used to print formatted text to the console. It is used here to print the header for the test results.
7. **$monitor**: This system task continuously monitors the specified variables and prints their values whenever they change. It is used here to print the values of a, b, and the outputs of the gates.
8. **#**: This symbol is used to specify a delay in simulation time. For example, #10 means a delay of 10 time units.
9. **$finish**: This system task terminates the simulation.

Compile and run:
To compile :

```
iverilog -o logic_gates_tb.out logic_gates_tb.v and_gate.v
or_gate.v not_gate.v xor_gate.v nand_gate.v nor_gate.v
```

To run :

```
vvp logic_gates_tb.out
```

```
PS C:\Users\gurra\Downloads\logic_gates> iverilog -o logic_gates_tb.out logic_gates_tb.v and_gate.v or_gate.v not_ga
te.v xor_gate.v nand_gate.v nor_gate.v
PS C:\Users\gurra\Downloads\logic_gates> vvp logic_gates_tb.out
Time | a | b | AND | OR | NOT(a) | XOR | NAND | NOR
   0 | 0 | 0 |  0  | 0  |   1    |  0  |   1  |  1
10000 | 0 | 1 |  0  | 1  |   1    |  1  |   1  |  0
20000 | 1 | 0 |  0  | 1  |   0    |  1  |   1  |  0
30000 | 1 | 1 |  1  | 1  |   0    |  0  |   0  |  0
logic_gates_tb.v:26: $finish called at 40000 (1ps)
PS C:\Users\gurra\Downloads\logic_gates>
```

Exercise:

1. Write Verilog code to implement a NAND gate and NOR gate.

2. Design a Verilog module that implements a 2-input XOR gate using only NAND gates. Write the corresponding code.

3. Create a Verilog module for a 2-input AND gate and another for a 2-input OR gate. Use them as components in a third module to implement a 2-input XOR gate.

4. Write individual Verilog modules for a 2-input OR gate and a NOT gate. Use them to implement a 2-input NOR gate in a third module.